

BUENOS AIRES HORIZONTE SOMBRÍO

RPG POR TURNOS // TEMÁTICA ARGENTINA

Trabajo Práctico Integrador — Fase A

Paradigma Orientado a Objetos // 2026

Integrantes:

Arce, Benjamín

Cruz, Ariel

González, Matías

Santolaria, Martín

1. CONCEPTO GENERAL DEL PROYECTO

Buenos Aires: Horizonte Sombrío es un videojuego RPG por turnos con elementos roguelite, centrado en combates sucesivos sin exploración libre del mapa. El jugador controla una party de entre dos y cuatro personajes que avanza a través de escenarios urbanos y rurales hasta enfrentar al jefe final en la Casa Rosada.

La propuesta mezcla folklore argentino con un entorno urbano moderno y decadente. Una niebla mística cubre el país, corrompiendo tanto a las criaturas del folklore como a la gente de la ciudad. Los protagonistas son los últimos Custodios de la Tradición, personajes que representan saberes y habilidades propias de la cultura argentina.

HISTORIA

El objetivo es viajar desde el campo hasta el centro de Buenos Aires para destruir el origen de la niebla, ubicado en la Casa Rosada, transformada en un tótem oscuro del poder corrompido. El jefe final es El Intendente del Olvido, una entidad con forma de burócrata gigante que concentra toda la corrupción del sistema.

ESTRUCTURA DE NIVELES

El juego se divide en cuatro niveles lineales, cada uno con combates sucesivos y un jefe al final. El primero transcurre en la pampa al atardecer —El Monte Herido— con enemigos como duendes y gauchos corrompidos, y el jefe Pomberito Corrupto, capaz de aturdir con un silbido. El segundo, El Cordón de Cemento, lleva la acción a una villa urbana deteriorada bajo la lluvia, con personajes corrompidos y enemigos armados. En el tercero, El Epicentro, la ciudad moderna presenta fuerzas organizadas con alta defensa y habilidades de escudo. Finalmente, el cuarto nivel culmina en la Casa Rosada enfrentando al jefe final con ataques que afectan a toda la party y efectos de aturdimiento y debilitamiento.

2. RELEVAMIENTO Y REQUERIMIENTOS FUNCIONALES

GESTIÓN DE PARTIDA

La aplicación permite iniciar una nueva partida o recuperar una guardada. El estado de la partida se persiste en archivo y puede ser recuperado desde la pantalla de inicio o desde el menú en batalla.

PARTY Y PERSONAJES

El jugador controla entre dos y cuatro personajes, cada uno perteneciente a una clase definida: Guerrero, Mago, Arquero o Curandera. Cada personaje almacena nombre, vida, maná (o recurso equivalente), ataque, defensa, velocidad, nivel y experiencia. Las clases pueden evolucionar al subir de nivel —Guerrero a Gaucho, Mago a Payador, Arquero a Boleador— con estadísticas mejoradas.

SISTEMA DE COMBATE

El combate es por turnos. El orden de acción se determina según la velocidad de cada entidad —personajes y enemigos—, replicando el sistema de Pokémon Rojo Fuego con elementos roguelite. En cada turno se puede atacar, defender, usar una habilidad o usar un ítem. Las habilidades consumen maná o inspiración según la clase. El sistema incorpora efectos de estado simples: aturdimiento (pierde el turno), defensa aumentada (escudo temporal) y debilitamiento.

ENEMIGOS

Los enemigos tienen estadísticas y comportamientos diferenciados según el nivel del juego. Existe un subtipo especial, EnemigoJefe, con más vida, ataques múltiples y habilidades propias. Al ser derrotados, los enemigos pueden soltar ítems que se agregan al inventario de la party.

INVENTARIO Y OBJETOS

El inventario administra tres tipos de objetos. Los consumibles —Torta frita, Mate, Alfajor de maicena— recuperan vida o maná en batalla y se obtienen como recompensa o drop de enemigos. Los equipables —Amuleto del Gauchito Gil, Amuleto de San La Muerte, Vino en cartón, Faconazo— modifican estadísticas del personaje de forma permanente o temporal, y se gestionan desde la pantalla de fogata entre niveles. Los objetos de misión son soltados por los jefes y desbloquean contenido narrativo.

PROGRESIÓN Y RECOMPENSAS

Al finalizar cada combate los personajes reciben experiencia obligatoriamente. Al subir de nivel, las estadísticas aumentan de forma automática. Los jefes sueltan ítems equipables o de misión que modifican atributos como ataque o defensa.

■ INTERFAZ GRÁFICA

La aplicación cuenta con cinco pantallas: inicio, estado del personaje/party, batalla, resultado y fogata. La pantalla de batalla tiene en la zona inferior izquierda los controles de acción —atacar, defender, habilidades, ítem— y en la superior el estado de la party. El menú de pausa permite guardar y cargar partida sin salir del juego.

3. CLASES PRINCIPALES DEL DOMINIO

La jerarquía central parte de Entidad, clase abstracta que agrupa los atributos comunes a personajes y enemigos. De ella descienden dos ramas: personaje (controlados por el jugador) y Enemigo (controlados por la IA). Los ítems forman una jerarquía paralela bajo la clase abstracta ítem.

JERARQUÍA DE ENTIDADES

Entidad *[abstracta]*

Raíz abstracta de todo participante del combate. Concentra atributos y comportamiento compartido.

atributos:

```
# nombre: String
# vidaMax: int
# vidaActual: int
# ataque: int
# defensa: int
# velocidad: int
```

operaciones:

```
+ atacar(objetivo: Entidad): void
+ recibirDanio(danio: int): void
+ estaVivo(): boolean
+ procesarEfectosDeEstado(): void
```

personaje *[abstracta]*

Extiende Entidad. Agrega progresión, inventario y habilidades propias del jugador.

atributos:

```
# nivel: int
# experiencia: int
# mana: int
# inventario: Inventario
# equipamiento: Equipamiento
# habilidades: List<Habilidad>
```

operaciones:

```
+ subirNivel(): void
+ ganarExperiencia(exp: int): void
+ usarHabilidad(h: Habilidad, objetivo: Entidad): void
+ defender(): void
```

guerrero

Especializado en combate cuerpo a cuerpo. Alta vida, ataque y defensa. Evoluciona a Gaucho.

atributos:

```
- bonusAtaqueCuerpoACuerpo: int
```

operaciones:

```
+ atacar(objetivo: Entidad): void
+ habilidadEspecial(): void
```

mago

Usa versos como habilidades. Recurso: inspiración. Habilidades: Aro aro aro (aturdir), Contrapunto (reducir defensa). Evoluciona a Payador.

atributos:

- inspiracion: int

operaciones:

- + usarHabilidad(h: Habilidad, objetivo: Entidad): void
- + atacar(objetivo: Entidad): void

arquero

Ataques a distancia. Puede reducir velocidad enemiga con Enredar. Evoluciona a Boleador.

atributos:

- rangoAtaque: int

operaciones:

- + atacar(objetivo: Entidad): void
- + enredar(objetivo: Entidad): void

curandera

Rol de soporte puro. Recupera vida y elimina efectos de estado. Habilidades: Curar el empacho, Tirar el cuerito, Curar gualicho.

atributos:

- poderCuracion: int

operaciones:

- + curar(objetivo: personaje): void
- + eliminarEfecto(objetivo: personaje): void

Enemigo

Extiende Entidad. Controlado por la IA. Elige acciones de forma autónoma según el contexto de batalla.

atributos:

- nivelDificultad: int
- dropItem: item

operaciones:

- + elegirAccion(): void
- + calcularDanioFisico(): int

EnemigoJefe

Versión reforzada de Enemigo. Más vida, fases y habilidades especiales como ataques a toda la party.

atributos:

- fasesDeVida: int
- habilidadesEspeciales: List<Habilidad>

operaciones:

- + habilidadEspecial(): void
- + cambiarFase(): void

SISTEMA DE ÍTEMS

item *[abstracta]*

Base abstracta para todos los objetos del juego.

atributos:

- # nombre: String
- # descripcion: String

operaciones:

- + aplicarEfecto(objetivo: personaje): void

ItemConsumible

Objetos de uso único. Recuperan vida, maná u otorgan efectos temporales.

atributos:

- cantidadCuracion: int
- tipoEfecto: String

operaciones:

- + aplicarEfecto(objetivo: personaje): void
- + usarItem(): void

ItemEquipable

Se equipan permanentemente modificando estadísticas.

atributos:

- bonusAtaque: int
- bonusDefensa: int
- bonusVelocidad: int

operaciones:

- + aplicarEfecto(objetivo: personaje): void
- + desequipar(objetivo: personaje): void

ItemMision

Objetos de trama soltados por los jefes. No se consumen ni equipan.

atributos:

- idMision: String

operaciones:

- + activar(): void

Inventario

Gestiona la colección de ítems de cada personaje.

atributos:

- items: List<item>
- capacidadMaxima: int

operaciones:

- + agregarItem(i: item): void
- + eliminarItem(i: item): void
- + obtenerItems(): List<item>

Equipamiento

Gestiona los ítems equipados actualmente, aplicando sus bonificaciones al personaje.

atributos:

- itemEquipado: ItemEquipable

operaciones:

```
+ equipar(i: ItemEquipable): void
+ desequipar(): void
+ getBonificacionDefensa(): int
```

HABILIDADES

Habilidad

Representa una habilidad especial ejecutable por un personaje o enemigo, consumiendo recursos cuando corresponde.

atributos:

```
- nombre: String
- costeMana: int
- danio: int
- efecto: String
```

operaciones:

```
+ ejecutar(emisor: Entidad, objetivo: Entidad): void
+ tieneSuficienteMana(e: personaje): boolean
```

MOTOR Y CONTROLADORES

MotorCombate

Núcleo del sistema de combate. Gestiona el flujo de turnos, calcula daño y verifica condiciones de fin de batalla.

atributos:

```
- turnoActual: int
- partyPersonajes: PartyPersonajes
- partyEnemigos: PartyEnemigo
```

operaciones:

```
+ procesarTurno(accion: String): void
+ calcularOrdenVelocidad(): void
+ calcularDanioFisico(emisor: Entidad): int
+ verificarFinBatalla(): boolean
+ distribuirExperiencia(party: PartyPersonajes): void
```

GameEngine

Controlador principal. Coordina transiciones entre pantallas y estados del juego.

atributos:

```
- estadoActual: EstadoPartida
```

operaciones:

```
+ iniciarNuevaPartida(): void
+ cargarPartida(): void
+ guardarPartida(): void
+ cambiarEstado(estado: String): void
```

ControladorBatalla

Intermediario entre PantallaBatalla y MotorCombate. Maneja los eventos de la interfaz de combate.

operaciones:

```
+ onAtacar(): void
```



```
+ onDefender(): void
+ onUsarHabilidad(h: Habilidad): void
+ onUsarItem(i: item): void
+ onBatallaTerminada(): void
```

controladorpartida

Gestiona guardado y carga del estado desde/hacia archivo.

atributos:

```
- estadoPartida: EstadoPartida
```

operaciones:

```
+ guardar(): void
+ cargar(): void
```

EstadoPartida

Contenedor serializable del estado de la partida.

atributos:

```
- nivelActual: int
- party: List<personaje>
```

operaciones:

```
+ serializar(): String
+ deserializar(data: String): void
```

PartyPersonajes

Agrupar a los personajes del jugador.

atributos:

```
- personajes: List<personaje>
```

operaciones:

```
+ getMasRapido(): personaje
+ distribuirExperiencia(exp: int): void
+ todosDerrota(): boolean
```

PartyEnemigo

Agrupar a los enemigos de la batalla actual.

atributos:

```
- enemigos: List<Enemigo>
```

operaciones:

```
+ todosDerrotados(): boolean
+ calcularExpTotal(): int
+ getMasRapido(): Enemigo
```

CrearEnemigo

Fábrica que instancia enemigos según el nivel actual del juego.

operaciones:

```
+ crearEnemigo(tipo: String, nivel: int): Enemigo
+ crearJefe(nivel: int): EnemigoJefe
```

INTERFAZ GRÁFICA

VentanaPrincipal

Ventana raíz. Contiene y coordina todas las pantallas.

atributos:

- pantallaActual: String

operaciones:

+ mostrarPantalla(nombre: String): void

PantallaInicio

Pantalla de bienvenida con opciones de nueva partida y cargar partida.

operaciones:

+ mostrar(): void

PantallaBatalla

Pantalla principal de combate con controles de acción y estado de party.

operaciones:

+ actualizar(): void

+ mostrarResultado(msg: String): void

PantallaPersonaje

Vista detallada de cada personaje de la party.

operaciones:

+ mostrar(p: personaje): void

PantallaResultado

Resumen al finalizar la batalla: experiencia, ítems obtenidos.

operaciones:

+ mostrar(exp: int, items: List<item>): void

PantallaFogata

Entre niveles. Descanso, inventario unificado y tienda.

operaciones:

+ mostrar(): void

4. RELACIONES ENTRE CLASES

Las relaciones principales del modelo son las siguientes. Las herencias reflejan la jerarquía es-un; las asociaciones, las dependencias de colaboración entre objetos en tiempo de ejecución.

HERENCIA

```
personaje —extends→ Entidad // personaje es una Entidad
guerrero / mago / arquero / curandera —extends→ personaje // cada
clase concreta extiende personaje
Enemigo —extends→ Entidad // Enemigo es una Entidad
EnemigoJefe —extends→ Enemigo // jefe es una versión especial de Enemigo
ItemConsumible / ItemEquipable / ItemMision —extends→ item //
jerarquía de ítems
```

ASOCIACIONES CLAVE

```
personaje —has→ Inventario // cada personaje tiene su inventario
personaje —has→ Equipamiento // cada personaje tiene equipamiento activo
personaje —has→ List<Habilidad> // los personajes conocen sus habilidades
Equipamiento —trabaja con→ ItemEquipable // aplica bonificaciones del
ítem equipado
MotorCombate —gestiona→ PartyPersonajes // el motor coordina la party
del jugador
MotorCombate —gestiona→ PartyEnemigo // el motor coordina la party
enemiga
MotorCombate —reporta a→ GameEngine // el motor notifica al engine
principal
ControladorBatalla —maneja→ PantallaBatalla // controlador de la vista
de combate
ControladorBatalla —delega en→ MotorCombate // delega toda la lógica al
motor
controladorpartida —persiste→ EstadoPartida // gestor de guardado y
carga
CrearEnemigo —fabrica→ Enemigo / EnemigoJefe // instancia enemigos según
el nivel
PantallaInicio —notifica a→ controlador menu // controlador de eventos
de pantalla
PantallaResultado —responde a→ GameEngine // el engine decide cuándo
mostrarla
```

5. PROPUESTA INICIAL DE INTERFAZ GRÁFICA

La interfaz se organiza en cinco pantallas con una estética oscura y urbana coherente con la temática del juego. Se describe a continuación la estructura y contenido de cada una.

PANTALLA DE INICIO

Fondo con imagen temática de Buenos Aires nocturno. El título del juego ocupa el centro superior. Debajo, dos opciones principales: Nueva partida y Cargar partida. En el borde inferior, nombre del equipo y la materia. Es la única pantalla completamente estática.

PANTALLA DE BATALLA

Es la pantalla más compleja del juego y está dividida en zonas. Al centro se representan visualmente los personajes a la izquierda y los enemigos a la derecha, cada uno con barra de vida. En la zona inferior izquierda, el panel de acciones: Atacar, Defender, Habilidades e Ítem. En la zona superior izquierda aparece el estado de la party con vida, maná y efectos activos. El menú de pausa (superior derecha) permite ver el estado completo, guardar y cargar partida. Al pie hay un log con el historial de acciones del turno.

PANTALLA DE ESTADO DEL PERSONAJE

Muestra la party completa con fichas individuales. Al seleccionar un personaje se despliega su detalle: estadísticas completas, inventario con opción de usar o equipar objetos, y el equipamiento activo con las bonificaciones que aporta.

PANTALLA DE RESULTADO

Aparece al finalizar cada batalla. Muestra un resumen: enemigos derrotados, experiencia otorgada a cada personaje e ítems obtenidos. Si algún personaje subió de nivel, se muestra una notificación. Al pie hay un botón para continuar hacia la fogata o el siguiente combate.

PANTALLA DE FOGATA (ENTRE NIVELES)

Pantalla de descanso entre niveles. Permite recuperar vida y maná parcialmente y revisar el estado de la party. Desde acá también se gestionan los ítems equipables: equipar, desequipar e intercambiar entre personajes. Los consumibles no se compran aquí; se obtienen exclusivamente como recompensas y drops en batalla. Un botón de avanzar inicia el siguiente nivel.

6. DECISIONES DE DISEÑO

HERENCIA VS. COMPOSICIÓN

Se eligió herencia para la jerarquía Entidad → personaje → clases concretas por la relación clara de es-un entre los elementos. Las habilidades y los ítems, en cambio, se modelan por composición para evitar la explosión combinatoria de subclases que aparecería si cada personaje tuviera sus propias versiones de cada objeto.

ARQUITECTURA MVC

El sistema sigue una separación Modelo-Vista-Controlador. Las clases de dominio —Entidad, personaje, Enemigo, ítem y derivadas— forman el modelo. Las pantallas son la vista. Los controladores —ControladorBatalla, controladorpartida, GameEngine— median entre ambos sin que el modelo conozca la interfaz gráfica.

MOTOR DE COMBATE CENTRALIZADO

El MotorCombate concentra toda la lógica de turnos, cálculo de daño y verificación de fin de batalla. Esto permite testear el núcleo del juego de forma completamente independiente de la interfaz, y facilita futuros cambios de reglas sin tocar las pantallas.

FÁBRICA DE ENEMIGOS

CrearEnemigo actúa como fábrica, desacoplando la instanciación de enemigos del resto del sistema. Agregar un nuevo tipo de enemigo no requiere modificar el motor ni los controladores, solo extender la fábrica.

PERSISTENCIA SIMPLE

El estado de la partida se serializa a través de EstadoPartida y es gestionado por controladorpartida. La lógica de guardado queda separada del modelo de dominio, lo que permite cambiar el mecanismo de persistencia —archivo de texto, JSON, binario— sin impactar al resto del sistema.

Arce, Benjamín // Cruz, Ariel // González, Matías // Santolaria, Martín

7. Apartado Visual

PERSONAJES:

Pombero:



Perro Corrupto:



Gaucha Corrupto:



Corrupto Armado:



Gaucha:



Curandera:



Arquera:



Mago Pampeano:



7. Apartado Visual

FONDOS DE NIVELES

El Monte Herido (Campo):



El Cordón de Cemento (Villa):



El Epicentro (CABA – Microcentro):



Casa de Gobierno (Nivel Final):

